Out of the Basement

An autoethnography of building Hewn, zero-knowledge payroll, with AI agents

Joshua Daniel Day
Independent; Hewn, hewnaccounting.com · hewnsupport@gmail.com
9 November 2026

ABSTRACT

I have worked in construction, agriculture and aviation. I never went to college. Between late 2023 and November 2026 I built Hewn, a free, open-source bookkeeping and payroll application whose payroll runs as private transactions on the Aleo zero-knowledge blockchain, with AI coding agents doing most of the typing. This paper is an autoethnography of that work. It asks how someone without a computer-science or cryptography background can direct AI agents to build security-critical software, and what had to grow up around the work to keep it honest. The data are three Git repositories (about 1,500 commits), a decisions ledger of 129 dated entries, a daily progress log signed by the sessions that wrote it, and the transcript of 1,059 of my own messages from the Hewn build. I report five findings: domain knowledge was the asset and computer science the thing acquired; the trades are in the method, a contractor's way of composing work, a farm's seasons and aviation's quality control; the human stayed the test instrument; memory had to be built as infrastructure because the agents have none; and the agents replaced the explanation a professor gives but not the judgement of one, which the project replaced with checks. The contribution is the record itself: every claim here points to a dated file in a public repository.

Keywords: autoethnography; AI coding agents; zero-knowledge proofs; Aleo; payroll; bookkeeping; end-user software engineering; safety culture.

1. Introduction

On 10 April 2026 a payroll run went through on the Aleo testnet in five transactions: an agreement was checked, a wage was paid in a stablecoin privately, receipts were minted for both sides, an audit record was anchored, and a cycle token was minted. Nothing on the public chain said who was paid or how much. I had written none of the code by hand, and I understood every step of it.

Those two facts together are what this paper is about. The code was written by AI coding agents (Claude Code sessions, Anthropic) working from documents I wrote and decisions I made, inside rules I set. The understanding came from three years of reading developer documentation, failing, and building. Neither alone would have produced a working system. Before 2025 the agents could not write Leo, Aleo's language, and kept writing Rust instead; and without the agents I could not have produced about 91,000 lines of TypeScript and 7,600 lines of Leo on the side of a full-time job.

Most writing about AI-assisted development studies programmers using agents to go faster. This paper is about the other case: a person with no programming background using agents to do work that would otherwise have required a degree and a team, in a field (zero-knowledge cryptography) where a mistake leaks wages. It asks three questions.

1. **What did the person contribute**, when the agent typed?
2. **What structures had to exist** for the result to be trustworthy, and what event made each one necessary?
3. **What did the agents replace, and what did they not**, compared with the education I did not have?

The method is autoethnography: I am both the subject and the instrument, and I write from a record. The record is unusually complete for this kind of study. Every merged pull request is dated. Every decision has a line that says who made it and where it is recorded. Every day of the Hewn build has a log entry signed with the id of the session that wrote it. And the session transcript holds every message I typed from the first day of Hewn to the day this draft was written. Section 4 says how I read it and what it cannot tell.

Section 2 says who is writing. Section 3 describes Hewn for a reader who knows neither Aleo nor bookkeeping. Section 5 tells the arc from 2023 to 2026 in five phases. Section 6 draws five findings from it. Section 7 says what this study cannot claim.

2. Who is writing

I have made my living with my hands and my attention: in construction, in agriculture, and now in aviation. I did not go to college. When I decided I wanted to understand zero-knowledge proofs, I looked at where that is taught and concluded I would need to be at MIT or somewhere like it to learn it from a professor. No community college was going to teach a tradesman to write proofs in a language a few years old. Each of those trades turned out to be in this project, and I set them out here so the reader can weigh what follows.

Construction came first. I started with landscaping and helping out on job sites, and the first thing it taught me was to keep going: to notice the small victories, a wall plumb, a grade that drains, that add up to a finished thing. Later I was leading jobs, and then running my own company as a general contractor, hiring subs. That is where I learned to think strategically: to break a job into tasks, hand each one to the right person, and make my intentions so clear that nobody had to guess. Not every job came with a set of plans. For custom work I often drew the blueprints myself, and found it was the part I loved most. Those years also taught me something I did not want to learn. I have always been good with numbers, and I still could not keep up with the books. The accounting software felt built for someone else; every screen assumed I already knew what it wanted. I closed that business. I have looked back more than once wishing there had been a better way to run payroll and keep the books for a crew. It was in those same years that I started reading about blockchains and trying to understand how they worked.

Agriculture taught me about the people in the fields: the ones grinding through twelve-hour night shifts, who matter more than any chart will show. I learned to work in big teams and to trust the people beside me when we could barely talk to each other across a language barrier, and to communicate anyway, with

hands and patience and whatever words we shared. We're all on the same bus. Nobody out here is better than anybody else. Farming also taught me that work has seasons, and that it is fine to put a thing down when its season is over and your attention is needed somewhere else. That is probably why I have stepped away from this project a handful of times, and why I have always come back to it.

Aviation is my work now, the first job I have had that was not swinging a hammer, driving a tractor or working with plants (I still get volun-told to do those things, because they know I'm good at them), and the first career with benefits I had never had. It taught me precision and quality control, and a standard of professionalism and customer service I had not worked to before. A good part of every day is inspections and the paper trail for them: daily, weekly, monthly, quarterly, biannual and annual checks on three trucks, two fuel farms, a de-ice truck and two passenger vehicles, every one written down, because the FAA audits all of it every year. You learn that a check is not done until it is documented, and that the paperwork is not separate from the safety. It is the safety.

Being a father gave me the reason. I want to build something that leaves things better for the next generation. Maybe my daughter starts a business one day and never has to dread payroll the way I did. Maybe she works for someone whose shop runs better because of something I built. Either would be enough.

Somewhere along the way I also learned what legal documents are for, when to protect yourself, and when to stop holding on to a thing so tightly. That is probably why Hewn is open source, and also why it carries a licence for collaboration and for commercial use: let it go, but write down the terms.

Three consequences for this study. First, I knew the problem from the inside. Hewn's payroll is for the kind of work I have done, hourly and seasonal, paid to crews, subs and vendors as much as to employees ("it's now not just to pay workers, we also have to consider paying vendors, or subs, and other businesses", 2026-10-14), and its first user is the contractor I used to be. Second, I ran the agents the way I ran a job: as the general contractor who composes the tasks, draws the plans and makes the intentions clear, then inspects the work (Section 6.2). Third, I work from a phone, between other things. Almost every message in the transcript was typed on one, which is why reports are short, detail goes in the log, and the agent writes the files.

I am the only participant in this study and its only author. The agents are not authors; they cannot be, and I am responsible for every claim here. Section 4 says how I separate what they wrote from what I said.

3. What Hewn is

Hewn is bookkeeping and payroll for a small business, free and open source under the AGPL. Its books are double-entry, like any accountant's. Three things make it unusual.

The books live on the business's own device, encrypted. There is no server and no database. The books sit in the browser's storage, encrypted with a key derived from a username and passphrase that never leave the device. Backups go to a file or, as encrypted letters, to the business's own account on the chain.

Money moves on a privacy blockchain. Hewn runs on Aleo, whose transactions are zero-knowledge proofs: the network verifies that a payment is valid without seeing the amount, the sender or the recipient. Workers are paid in USDCx, a stablecoin, by private transfers. The public chain holds only hashes, commitments and block heights. A paystub travels as a sealed record, a "letter", readable only by the two parties.

Closed periods are provable. When a business closes a month, one fingerprint of every entry is anchored on the chain. The business can hand an accountant or a lender a proof, and they can check it against the chain without seeing the books. The technical paper [Day 2026a] describes the cryptography; this paper does not repeat it.

Every person on Hewn has a registered name (`name.hewn`) tied to a Shield wallet, and a wallet is required to use the app at all. Businesses, workers and customers each have their own kind of account, and a business and a person never share a wallet.

The codebase on 2026-11-09: about 91,000 lines of TypeScript in the app and its packages, 7,610 lines of Leo across sixteen live programs on testnet, 1,442 automated tests in 172 files, 27 design documents, and 33 written rules of which 21 are enforced by a check in continuous integration. It has never run on mainnet, and I have said I may never take it there myself (Section 6.5).

4. Method

This is an autoethnography in the analytic sense [Anderson 2006; Ellis, Adams and Bochner 2011]: a member of the setting writes about it from the inside, with the aim of explaining something beyond the self. The setting is a one-person software project with AI agents as the other workers. I am its only human member.

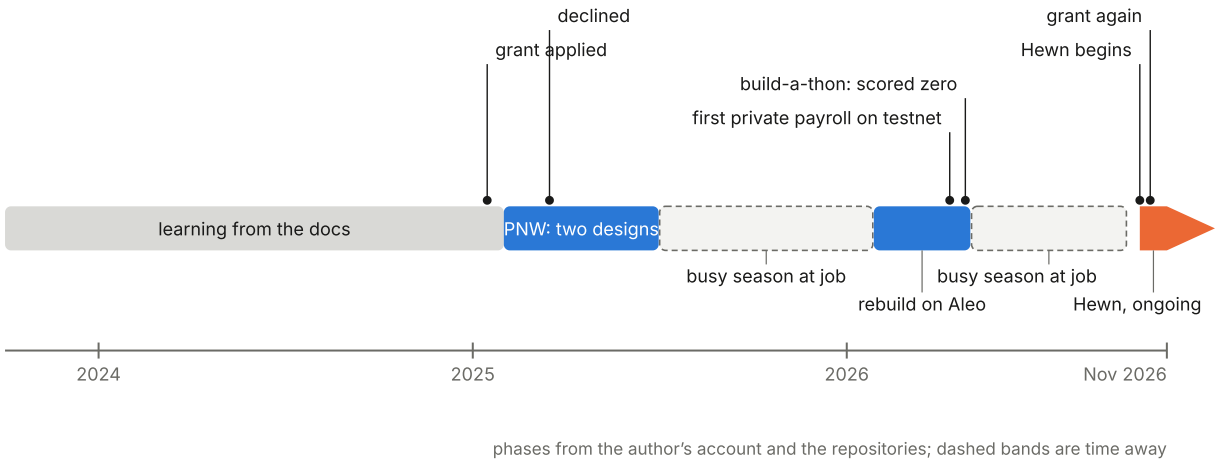
Data. Five sources, all dated.

1. Three Git repositories: `pnw_mvp_v2` (the Leo programs, from 2026-01-27, 629 commits), `pnw_employment_portal_v1` (the first app, from 2026-03-14, 450 commits, retired 2026-10-13) and `hewn` (from 2026-04-11 as a submission repository; the Hewn build from 2026-10-13; about 280 merged pull requests).
2. The decisions ledger, `memory/decisions.md`: 129 entries, each with a date, the decision, who made it, where it is enforced and where it is recorded. 123 name me as the decider.
3. The lessons file, `memory/lessons.md` (38 entries), and the daily progress log (25 days, 299 entries, each ending with the id of the session that wrote it).
4. The transcript of the Hewn build session, 2026-10-13 to 2026-11-09: 1,059 messages of mine, 34,292 words, about eighteen words each.
5. The pre-2026 history as I told it for the project README, and the two grant applications, January 2025 and October 2026, whose proposal documents are kept in the repository.

The co-writing problem. Most of the written record was written by the agents, not by me. The progress entries, the ledger lines, the lessons and the pull request descriptions were composed by a session at my direction and often from my words, but they are the agent's prose. In an autoethnography the author is the instrument, so this matters. I treat the agent-written files as field notes we co-wrote, and I say so wherever I quote one. My own voice is in the transcript: every message there was typed by me. The quotations in Sections 5 and 6 are drawn from it, each with its timestamp.

Analysis. I read the record in three passes. First a timeline: every milestone, decision and incident placed by date (Appendix A). Second, for each structure the project now has (the rules with ids, the approval rule, the memory scopes, the sleep cycle), the event that made it necessary, found by reading backwards from the ledger line to the progress entry to the transcript of that day. Third, my own messages, read for what I contributed that the agent did not: corrections, refusals, designs, and the questions I asked.

5. The arc, 2023 to 2026



phases from the author's account and the repositories; dashed bands are time away

Figure 1. The arc, late 2023 to November 2026: phases of learning, building and time away, with the milestones the text turns on. Phases follow the author's account; dates of milestones are from the repositories.

5.1 Learning without a teacher (late 2023 to 2024)

I started reading about Aleo around the time it opened to builders, late in 2023 and into 2024. The developer documentation was the only teacher: transitions, asynchronous finalize code, and the way Leo compiles to Aleo's own instruction language. The farming season of 2024 took the hands-on building for the summer and fall, but not the learning. In the spare hours, podcasts and research papers took the place of television and video games, and they have kept that place since. In my words, from the transcript:

> "there is no community college that is going to teach someone how to write Zero Knowledge proofs on a language that was just made public a few years old. In the early stages, AI wasn't even much help because it was so new that AI had no database to pull from. It kept writing things in Rust instead of Leo." (2026-10-15)

It wasn't until late 2025 that the proofs themselves started to make sense, and it was the old teaching examples that did it. First the colour-blind friend: you hold a red ball and a green ball that look identical to them; they hide the balls, swap them or not, and you say whether they swapped. Once could be a guess; twenty times in a row and they are convinced the balls differ, without ever learning which is which. Then the deck of cards: to prove the card I drew is red without showing it, I turn over the twenty-six black cards still in the deck, and you know mine is red without knowing which red card it is. Then the three-coloured map: I colour a map so that no two neighbours share a colour, cover it, and let you uncover any one border; the two sides always differ, and after enough borders, the colours reshuffled each time, you believe the map can be coloured without ever seeing how. Each example is the same move: convince someone a statement is true while showing them nothing they could reuse. Aleo's records are that move applied to a payment.

5.2 Learning by building: Proven National Workers (2025)

In February 2025 I decided the way to learn was to build something real. The first design was enormous: a union-style network for citizen and immigrant workers, with on-chain payroll, worker-run local DAOs, farm co-ops, trust funds, a marketplace and bridges to three other chains. Hundreds of commits of Leo went in during the first month, across repositories whose names still show the size of the ambition (`pnw_mvp`, `Proven-National-Worker`, `pnwxkd`). I had applied for a grant in January, on the strength of what I had built while learning the year before; in March the answer came back no, because the project was not built out far enough and was not on testnet. The second version that spring kept two chains, Aleo for private payroll records and Stellar for payouts, because Aleo had no native stablecoin yet and I was trying to bridge to Stellar to pay in USDC, and passed a thousand commits; my GitHub graph for 2025 shows 1,736 contributions, nearly all of them between February and June. Then, from June 2025 to February 2026, I put it down. Work needed my focus, and I felt a little defeated: I just didn't know enough yet. I was trying to build my own off-ramp into Stellar to make the payments, with a DAO-run "gas station" to swap tokens into a stablecoin, and it was too much. Part of me figured: wait until the technology catches up. Part of me felt I was in way over my head, learning two blockchain languages at once. Every cut, and the time away, made the core clearer: private, honest payroll.

5.3 Starting over on Aleo alone (January to April 2026)

On 2026-01-27, after eight months away, I started `pnw_mvp_v2` clean: Aleo only, USDCx (Aleo had just announced USDCx, and I felt the spark: finally, this was what I had been waiting for), one adapter as the only path to the chain. The first day's commits are architecture documents; the programs follow from the next day. Two rules from that winter survive in Hewn's ledger today: adapters are the only execution boundary (one module is the only road from the app to the chain and everything else only plans, so there is one place to audit how a transaction is built and signed; it is why the wallet could later become the only signing path without a hunt through the code) (2026-02-27), and public mappings never hold cumulative counters, because they leak wages by difference (2026-03-02).

The portal for employers and workers began on 2026-03-14, again with documents first. The first day's nine commits are design files whose messages are the GitHub web editor's defaults, I uploaded them by hand before any code existed. Somewhere in those weeks I also started running the agent on a desktop as well as from my phone, which may be why; I can't place the date, and I have since gone back to the phone. March was the busiest month of the project to that point: 320 commits on the programs and 258 on the portal.

On 2026-04-10, in the last wave of a build-a-thon, the first private payroll ran end to end from the portal (Section 1). Three workers followed the next day, credential tokens the day after, and a tax engine with W-4 forms and timesheets within the week. Everything was finally on testnet and working well, and on 2026-04-11 I made a submission repository to hold both halves of the project. Then I put the wrong Vercel address in the application form. The judges opened a link to nothing, and the project scored zero. It was heartbreaking: a zero on the SAT because I forgot to write my name on it. The five transactions are still on the chain. The scoresheet never saw them, or the website I had built for all of it.

Then, from May to September 2026, almost nothing: four commits in five months. After the build-a-thon I stepped away for the busy season at work, as I had the year before. That is a season, not a stop (Section 6.2). The project waited where I had left it, and in October I picked it back up.

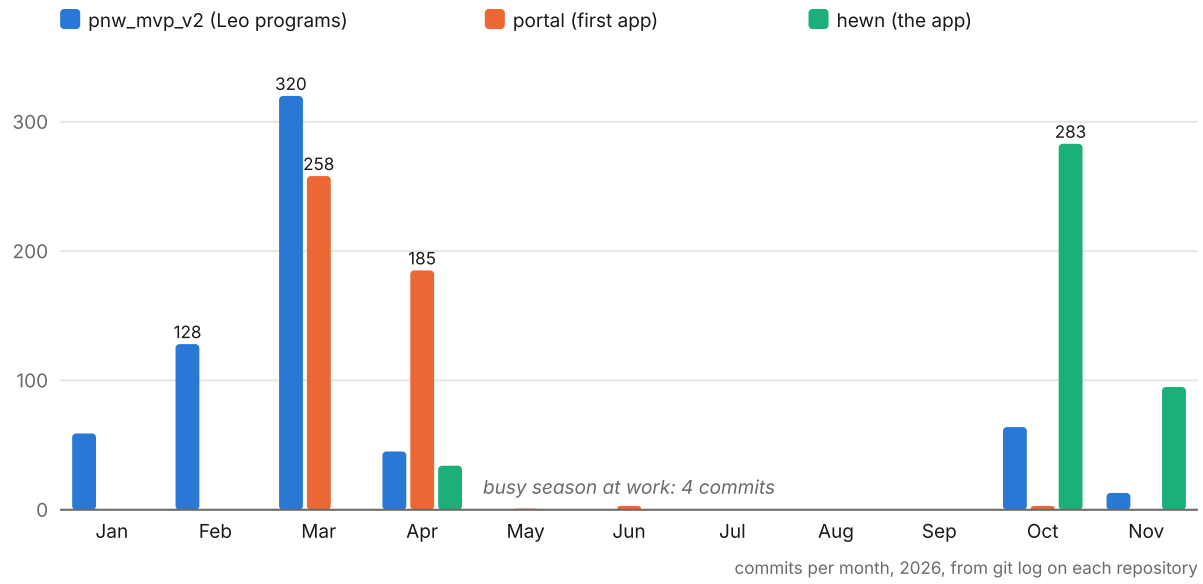


Figure 2. Commits per month in 2026 across the three repositories. The shape is the seasons of Section 6.2: the rebuild from late January, the build-a-thon in April, four commits from May to September, and Hewn from 13 October. The 2025 work (1,736 contributions, nearly all February to June) is on the earlier repositories named in Section 5.2.

5.4 The pivot to Hewn (2026-10-13 to 10-15)

On 2026-10-13 I attached two documents I had written, a build plan and a rules file, and asked for bookkeeping and payroll built the way bookkeepers already think, on top of the payroll stack. The first day's log has thirteen entries: the portal's libraries were salvaged into packages, and a shell, an encrypted vault, a ledger core, payroll posting, financial statements, and customers, vendors and invoices were built,

all in “fiat” mode, by the end of the day. My first message set the tone: “Don’t overthink either of them just yet, let’s focus on building a working clean room with Fiat.” My questions that afternoon were the ones a careful outsider asks: “Is TS going to be secure? How can we ensure people can’t hack into it?” They were basic questions, and I knew it; what I wanted was assurance that the way it was being written in TypeScript was secure.

The rules were there from the first day too: no keys in the repository; browser storage is not a book of record; anything tied to a person is private in every mode; one task per pull request; and any change to a Leo program said out loud first, so the session could be switched to the stronger model. The product was named Plumb that evening (“Wait pause.... Plumb is already taken”) and Hewn the next day.

Two decisions in those three days were mine, and they changed the architecture. On 10-14 I asked: “What if we only have a single transition. Really we only need it as a payment rail that can be referenced privately.” I had realised the transaction did not need to carry everything; that is the joy of a zero-knowledge proof. The money could move on its own, and an encrypted record about the payment could carry the rest. That became the design: one private stablecoin transfer per worker, the paystub sent as a letter, and the elaborate payroll program left deployed but unused. I had said “fiat first” on day one, but the wallet was always where this was going; building the books first in fiat mode, like any first accounting app, was a way to get the bookkeeping right before the chain was plugged in. On 10-15, after testing how the wallet’s own history looked in the app, the order changed and the wallet became the way in. On 11-03 the point was settled: “a user can’t even use the software at all [without a Shield account]; being able to track Fiat is a bonus.”

5.5 The structures (October to November 2026)

The rest of October built outward: letters as sealed records, the chain as the mailbox (10-16 and 10-18); a public job board (10-19); a two-curve key exchange I named “twin-curve“ that keeps every conversation readable by two people alone (10-20); it shares a word with Dual_EC_DRBG, the backdoored random-number generator, but nothing else: no second point, no hidden relationship, two standard constructions on two curves, and every key made on the person’s own device; a two-party agreement program (10-24); credentials and cards (10-25). Twenty-three programs are now listed as deployed on testnet.

On 10-23, ten days into the build, I applied for a grant again, with the system live on testnet. That application points at the repository and the live site rather than at a frozen document, so what the reviewers see is whatever is current on the day they look, if they have looked. At the time of writing I have not heard back.

From 10-25 the work turned to making the project trustworthy, and each structure has a trigger in the record.

- **Rules with ids, tied to checks** (10-25). A plan to be “audit-ready” gave every rule an id, a place where it is enforced and a threat it answers, and turned every rule that had no check into one. Twenty-one of twenty-four now have a check in CI.
- **An incident write-up** (10-27). The app stuck on “Opening…” in two browsers and recovered by itself. The cause was never found; the write-up says what was ruled out and what to do if it recurs.

- **Approval in chat** (11-03). “As long as I approve them here in chat, on this specific device” became the rule that nothing merges without my word, and the merge commit says so.
- **Memory as files** (11-03). Decisions and lessons were consolidated into a ledger, and a weekly pass (a “dream”) was set to verify and prune it, because a session forgets everything when its context fills.
- **The sleep cycle** (11-07). After a session lost state in a mid-task compaction, consolidation was moved to every rhythm: before each compaction, nightly, weekly, and a monthly pass that practises on random pairs of the codebase and keeps only what finds something.
- **One sleep pull request at a time** (11-09). The first weekly pass repeated four decisions the nightly pass had already filed, because it could not see an unmerged pull request. The fix was written the same day.

In the same week the phone-browser path was made to work over the wallet’s relay, found and fixed through my own tests (Section 6.3), and the three-worker payroll was rehearsed end to end against a local Aleo node in CI (11-08).

6. Findings

6.1 Domain knowledge was the asset; computer science was acquired

The agent knew how to write a double-entry ledger. It did not know that a crew is paid differently from an employee, that a sub sends an invoice, that a wallet on a phone cannot approve a batch of payments (“No their wallet cannot approve batched payroll payments”, 2026-10-15), or what a business owner will and will not tolerate at the end of a day. Those came from me, and they are the decisions the ledger records: 123 of 129 name me. The computer science came the other way, through questions (“explain the difference between a letter and a page, why we have 2 different, and is that something that Leo enforces?”, 10-16).

6.2 The trades are in the method

Three habits from three trades show in the record, each adopted by my decision on the day it was needed.

A contractor’s composition. Every start in the record (2026-01-27, 2026-03-14, 2026-10-13) was documents first and code second: an architecture, a build order, a rules file, the way a custom job starts with blueprints I had to draw first. The working agreement’s first rule, one task, one exit criterion, one pull request, is a sub’s scope of work. The build plan’s phases with exit criteria are a schedule. And I directed the agents the way a general contractor directs subs: compose the task, make the intention clear, inspect the result, and never let a sub change the plans on their own (“Any new or changed Leo program is said out loud first”, 2026-10-13). I was not a programmer among programmers. I was the GC.

A farm’s seasons. The gaps in the record, the summer of 2025 and May to September 2026, are not abandonment. Farm work taught me that work has seasons and that attention goes where it is needed; the project rested when work’s season came and picked up where it had left off. The README says this plainly, and so should the paper. The same years taught me to trust a team across a language barrier and to communicate anyway, which is not a bad description of directing an agent whose vocabulary is not mine.

Aviation's quality control. At work, a good part of every day is inspections and the paper trail for them: daily, weekly, monthly, quarterly, biannual and annual checks on three trucks, two fuel farms, a de-ice truck and two passenger vehicles, each one written down because the FAA audits the lot every year. Hewn's second month built the same thing in software. Every rule has an id, a check and a threat it answers, and the list of them is kept as the control list for an audit that has not happened yet; an internal one stands in until it does. Every day's work ends in a signed log entry. Nothing merges without a second set of eyes, and when something went wrong and could not be explained, it was written up anyway. The sleep cycle runs on a cadence I recognised at once: a pass at every compaction, nightly, weekly, and a monthly one that practises on random pairs of the codebase. The design came from a research paper on memory, but the rhythm came from the hangar, where a check is not done until it is written down.

6.3 The human stayed the test instrument

Four defects in the week of 2026-11-08 were invisible in the code and found on my phone: a wallet connection that silently stayed stale after the first trip to the wallet app; a backup the browser refused to share because of its file type; a name registration the wallet reported as failed when the chain had accepted it; and a restore screen that waited on a wallet that was asleep. Each was found by using the app as a business would, and each fix was confirmed the same way. The agent also mis-stated the record that week, saying the restore took one trip to the wallet, and I corrected it: "Technically there was two trips to shield" (2026-11-08). The lesson the project wrote from that period is that anything touching the wallet, the relay or a browser's permissions needs a test on a device, not a review of a diff.

6.4 Memory had to be built, because the agents have none

A session's memory ends when its context fills. The project's answer was to put memory in files and to schedule work on them: a ledger of decisions, a file of lessons, a daily log, and passes that run on a timer to move what was learned up a ladder, from the day's log to the ledger and, by my decision only, into the rules. The passes are themselves a source of error: the first weekly pass duplicated the nightly one. But the alternative, shown on 2026-11-07, is a session that moves a working tree back to an old commit because the fact that would have stopped it was in the part of the context that had been summarised away. Memory here is infrastructure, like tests. It is also a paper trail, the kind an inspector can follow from a rule to the check that proves it to the day it was decided and the session that decided it.

6.5 What the agents replaced, and what they did not

The agents replaced the patient explanation a professor gives, on demand, at two in the morning, and they replaced the team I did not have. They did not replace a professor's judgement about whether I had understood, nor a credential, nor peer review. The project's substitute for judgement is mechanical: 1,442 tests, a CI that blocks on the invariants, a payroll rehearsal against a real node, and a rule that every program that will hold real money gets a third-party audit before mainnet. I am also clear about where this stops. I have said I may never take Hewn to mainnet myself: "have it fully functional, sitting within testnet ready for someone else to bring it to mainnet, or if I get grants" (2026-11-07). That is not modesty. It is the line between what I can verify and what I cannot.

A last observation, about cost. The constraint on the build was not the agent's ability but my usage allowance and my hours: "Ok please continue my limit just reset" (10-25). The busiest day of the transcript, 10-24, was a Saturday, spent at home, and has 118 of my messages.

7. Limitations and ethics

One participant. This is one person's account of one project. It generalises as an existence proof and a description, not as a measurement.

Co-written field notes. The logs were written by the agent at my direction. I use them for dates and facts, which the repositories confirm, and only my own transcript for what I thought and wanted. Where a ledger line attributes a decision to me, the transcript was checked for the message behind it; the weekly pass of 2026-11-09 itself flagged one attribution it could not confirm.

The record begins on 2026-10-13. My own words before the Hewn build survive only in the README's account and in commit messages. The 2025 phases rest on my recollection.

No users. Hewn runs on testnet with test money. No real wage, name or address has passed through it, and none appears in this paper.

Interest. I own the project and would benefit if it were funded. The grant proposals are in the repository.

AI and authorship. The agent helped prepare the draft of this paper from the record. Anyone checking a claim can follow its date to the file.

8. Conclusion

A person from the trades, without a degree, built a working private payroll and bookkeeping system on a zero-knowledge blockchain in his spare hours, with AI agents writing the code. The record shows what that took: two years of reading before the agents were useful; a problem known from the inside; the willingness to cut a huge design down to the part that mattered; and, once the code existed, a set of structures built one incident at a time so that the work could be trusted by someone who could not personally verify the mathematics. The agents did the typing and the explaining. The judgement, the direction, the testing on a phone at the end of a shift, and the decision about where to stop were a person's.

The contribution is the record. Every date here points to a file in a public repository, and the structures described (rules with ids and checks, approval in chat, memory as files, sleep passes) can be read and copied. Once, I built a working system and scored zero because the judges could not find it. The project that came after writes everything down, the way an inspection on a fuel truck is not finished until the form is signed. What I would want another person in my position to take from this is simpler than any of the cryptography: the documentation was enough to learn from, building was the way to learn, and what made the result trustworthy was not cleverness, it was just a paper trail.

References

Anderson, L. (2006). Analytic autoethnography. *Journal of Contemporary Ethnography*, 35(4), 373–395.

Ellis, C., Adams, T. E., and Bochner, A. P. (2011). Autoethnography: an overview. *Forum Qualitative Sozialforschung / Forum: Qualitative Social Research*, 12(1), Art. 10.

Ko, A. J., Abraham, R., Beckwith, L., Blackwell, A., Burnett, M., Erwig, M., Scaffidi, C., Lawrance, J., Lieberman, H., Myers, B., Rosson, M. B., Rothermel, G., Shaw, M., and Wiedenbeck, S. (2011). The state of the art in end-user software engineering. *ACM Computing Surveys*, 43(3), Article 21.

Behrouz, A., Hashemi, M., Javanmard, A., and Mirrokni, V. (2026). Language Models Need Sleep: Learning to Self-Modify and Consolidate Memories. arXiv:2606.03979. The sleep framework Hewn's consolidation cycle is modelled on.

Day, J. D. (2026a). *Hewn: private payroll and provable books on Aleo.* Technical paper v1.0, draft. `docs/PAPER.md` in the Hewn repository; to appear at hewnaccounting.com/paper.

Day, J. D. (2026b). *Grant proposal: Proven National Workers* (April 2026) and *Grant proposal: Hewn*. `docs/grants/` in the Hewn repository.

Aleo developer documentation. https://developer.aleo.org

GNU Affero General Public License, version 3. https://www.gnu.org/licenses/agpl-3.0.html

Repositories: github.com/PNWBNW/hewn, github.com/PNWBNW/pnw_mvp_v2, github.com/PNWBNW/pnw_employment_portal_v1.

Appendix A. Timeline of milestones, with sources

“decisions” is `memory/decisions.md`; “progress” is `progress/<month>/<date>.md`; “transcript” is the build session’s transcript, 2026-10-13 to 2026-11-09; “author” is the author’s own account given on the date shown.

Date	What happened	Source
late 2023 to 2024	Learning Aleo from the developer docs alone; AI assistants not yet useful for Leo (they wrote Rust)	README; transcript 2026-10-15
2024, summer and fall	Hands-on building paused for the farming season; the learning didn’t: podcasts and research papers in the spare hours	author, 2026-11-09
2025-01	First grant application, on what had been built while learning	author, 2026-11-09
2025-02	PNW’s first design: a union-style worker network with on-chain payroll, local DAOs, farm co-ops, trust funds, a marketplace, bridges to three chains; hundreds of Leo commits in a month	README; GitHub
2025-03	The grant declined: not built out far enough, not yet on testnet	author, 2026-11-09
2025, spring	Second version: Aleo for private payroll, Stellar for payouts, NFT paystubs; more than 1,000 commits; 1,736 GitHub contributions in 2025, nearly all February to June	README; GitHub
2025-06 to 2026-02	Away: work needed the focus, and “I just didn’t know enough yet”	author, 2026-11-09
2026-01-27	pnw_mvp_v2 begins: Aleo only, USDCx, one adapter as the only path to the chain; the first day is architecture documents, programs from the next day	git
2026-02-27	Rule: adapters are the only execution boundary	decisions ledger
2026-03-02	Rule: no cumulative counters in public mappings (they leak wages by difference)	decisions ledger
2026-03-14	The employer and worker portal begins: nine design documents on day one, code from 03-15	git
2026-03	Busiest month before Hewn: 320 commits on the programs, 258 on the portal	git
2026-04-10	First private payroll on testnet, five transactions, in the last wave of a build-a-thon	pnw_mvp_v2 README
2026-04-11	Three-worker payroll; a submission repository is created (it becomes hewn)	portal CLAUDE_2026-04.md; git
2026-04	The build-a-thon submission scores zero: the wrong Vercel address on the form, the judges could not open it	author, 2026-11-09
2026-04-12 to 04-20	Credential tokens; tax engine, W-4, timesheets, pay rates	portal CLAUDE_2026-04.md
2026-04-20	Grant proposal drafted (PNW version); a Hewn version follows later	docs/grants/
2026-05 to 2026-09	Four commits in five months: the busy season at work	git; author, 2026-11-09
2026-10-13	Hewn begins: a build plan and a rules file attached; 13 log entries; Phases 0 to 4 built in fiat mode in one day; the first rules	progress 10-13; transcript
2026-10-14	Named Hewn (Plumb was taken); one private USDCx transfer per worker, paystub as a letter	decisions; transcript

Date	What happened	Source
2026-10-15	A wallet is required to get in; AGPL-3.0; open registration on chain; the vault scheme; one kind of account per wallet	decisions
2026-10-16, 10-18	Letters as records on chain (hewn_letters_v1, then v2): the chain is the mailbox	pnw_mvp_v2 CLAUDE.md
2026-10-19	The public job board (hewn_jobs_v1)	same
2026-10-20	Twin-curve key exchange so every conversation is readable by two people alone	transcript; technical paper
2026-10-23	Second grant application, pointing at the repository and the live site; no answer at the time of writing	author, 2026-11-09
2026-10-24	employer_agreement_v5 deployed; the busiest day of the transcript, 118 messages	manifest; transcript
2026-10-25	Credentials and cards deployed; the audit-ready plan: rules with ids tied to checks and threats; feed, reconcile, seal; a third-party audit before mainnet	decisions
2026-10-26	Reconciliation; 36 log entries, the most in a day	progress
2026-10-27	Incident: stuck on “Opening…”, recovered on its own, cause unknown, written up	docs/incidents/
2026-11-02	Close a period	decisions
2026-11-03	The app runs on Aleo, fiat a bonus; approval in chat before any merge (OPS-01); memory in three scopes and a weekly dream; the map; a refused command stays refused; SOC 2 framing	decisions; transcript
2026-11-05	Period proof and its public check (/verify); technical paper v1.0 draft	BUILD_PLAN.md; docs/PAPER.md
2026-11-07	The sleep cycle (hooks, skills, REM); licensing (NOTICE, 7(b) term, commercial licence, CLA); October closed and anchored on testnet	decisions; progress
2026-11-08	Phase 2 rehearsed on a local Aleo node in CI; adapters 1.5.0; a plain phone browser over the relay; backups shared as text; released	progress 11-08
2026-11-09	The first weekly dream repeated the nightly pass; one sleep pull request at a time	progress 11-09

Appendix B. The record in numbers, 9 November 2026

Measure	Value	How it was counted
GitHub contributions	1,736 in 2025; 2,612 in 2026 to 11-09	the author’s contribution graphs
Commits, three repositories	629 + 450 + 412	git log on each main
Merged pull requests on hewn	283 (plus 6 in April)	squash-merge titles (#N)
Decisions in the ledger	129, of which 123 name the author as the decider	memory/decisions.md
Lessons	38	memory/lessons.md

Measure	Value	How it was counted
Progress log	25 days, 299 entries, each signed with a session id	progress/
The author’s messages, 2026-10-13 to 11-09	1,059; 34,292 words; about 18 words each; 103 approve or merge something	transcript
TypeScript in the app and packages	about 91,000 lines	wc -l on tracked .ts/.tsx
Leo	7,610 lines in 16 live program folders; 23 deployed program entries on testnet	pnw_mvp_v2/src, manifest
Automated tests	1,442 in 172 files	the 2026-11-08 progress entry
Design documents	27 with frontmatter and a generated index	docs/
Written rules	33 (24 invariants, 9 operating rules); 21 invariants have a CI check	docs/INVARIANTS.md
Incidents written up	1 (2026-10-27)	docs/incidents/
Deployed to mainnet	nothing	

Source: the Hewn repositories, github.com/PNWBNW/hewn, github.com/PNWBNW/pnw_mvp_v2 and github.com/PNWBNW/pnw_employment_portal_v1. The decisions ledger, the lessons, the progress log and the design documents cited here are files in the first of them. © 2026 Joshua Daniel Day. This paper is licensed under CC BY-NC-ND 4.0 (https://creativecommons.org/licenses/by-nc-nd/4.0/): it may be shared with credit, not sold or altered. The code it describes is AGPL-3.0-only.